```
"""
==============================================================
Sprint 1 - Task 5 (Group A): Energy Data Lead - Exploratory Data Analysis (EDA)
CeDInt building, Storey 1 - energy meter data ( BM321 / BMT01)
==============================================================

WHAT DOES THIS SCRIPT DO?
It reads the four Excel exports (one file per zone NE/NW/SE/SW, one sheet per
device category such as HVAC, Lighting ...) and analyses ONLY the electrical
quantities measured by the energy meters:
    W  = active power (instantaneous)          I = current
    Ea = active energy (cumulative counter)    Er = reactive energy (counter)
    Pa = apparent power
The ambient sensors (Lum, Hum, Temp, Pres) are NOT part of Task 5 and are
only checked for availability.

OUTPUT
- tables (CSV) and figures (PNG) in the folder "eda_output"
- an automatically generated summary "00_key_findings.txt"
- an hourly time series "hourly_energy.parquet" for Node A / Node B

HOW TO RUN IN PYCHARM
1. Terminal:  pip install pandas numpy matplotlib seaborn python-calamine pyarrow openpyxl
2. Put a folder "data" next to this file and copy the 4 Excel files into it
3. Click the green ▶ next to  if __name__ == "__main__":
==============================================================
```

Imports & Config

File - /Users/emilywiesner/PycharmProjects/PythonProject/main.py

```
26 ============================
27 """
28
29 # ------------------------------------------------------------
   -------------------------
30 # Imports
31 # ------------------------------------------------------------
   -------------------------
32 import glob                          # find files by
   pattern (e.g. *.xlsx)
33 import os                            # paths and
   folders
34 import re                            # regular
   expressions (read zone from file name)
35 from pathlib import Path             # OS-independent
   paths
36
37 import matplotlib    plots
38 import matplotlib.pyplot as plt  # plotting
39 import numpy as np                   # numerical   maths
   computations
40 import pandas as pd                  # tables (
   DataFrames)
41 import seaborn as sns  plots         # statistical
   plots with nicer defaults
42
43 # ============================ CONFIGURATION
    ================================
44 # Change settings here without touching the rest of
    the code.
45 BASE_DIR = Path(__file__).resolve().parent   #
   folder containing this script
46 DATA_DIR = BASE_DIR / "data"     joins paths  #
   folder with the Excel files
47 OUT_DIR = BASE_DIR / "eda_output"            #
   folder for all results
48
49 SHOW_PLOTS = True          # True  = show all figures
    once at the END of the run (and save them)
50                            # False = only save PNGs (
   no windows)
```

folder where script is in

```
51 GAP_THRESHOLD_MIN = 180  # a pause longer than this
    (minutes) counts as a "gap"
52 MIN_HOURS_PER_DAY = 20   # days with fewer hours
   are ignored for daily energy
53 OUTLIER_Z = 6            # threshold for the robust
    z-score (outliers)
54 # ================================================
   ========================
55
56 ENERGY_COLS = ["W", "I", "Ea", "Er", "Pa"]   #
   electrical quantities (our focus)
57 SENSOR_COLS = ["Lum", "Hum", "Temp", "Pres"] #
   ambient sensors (not part of Task 5)
58
59 # Consistent look for all figures
60 sns.set_theme(style="whitegrid", context="notebook"
   )
61
62 # Without display we use the "Agg" backend: figures
    go to files only, no windows.
63 if not SHOW_PLOTS:
64     matplotlib.use("Agg")
65
66 # Key findings collected during the run (written to
    00_key_findings.txt)
67 FINDINGS: list[str] = []
68
69
70 def note(text: str):
71     """Store a finding and print it to the console
   ."""
72     FINDINGS.append(text)
73     print(f"  -> {text}")
74
75
76 def finish(fig, name: str):
77     """Save a figure as PNG. Figures are NOT shown
   here - if SHOW_PLOTS is True,
78     all of them are displayed together at the end
   of main()."""
79     fig.tight_layout()
```

incomplete days would look like gaps

↳ one plot style for all figures

no windows.

adds finding to the list & print it

stops labels from overlapping

```
80      fig.savefig(OUT_DIR / name, dpi=130)
    # always save as PNG
81      if not SHOW_PLOTS:
82          plt.close(fig)
    # free memory if nothing is shown later
83
84
85  # ================================================
    ============================
86  # 1) INGESTION: every sheet of every file -> one
    long table
87  # ================================================
    ============================
88  def load_all(data_dir: Path) -> pd.DataFrame:
89      """Read all 'Storey1_<ZONE>_report_by_device.
    xlsx' files.
90
91      Each file = one zone, each sheet = one device
    category.
92      Result: one DataFrame with the extra columns '
    Zone' and 'Category'.
93      Excel is slow, so the first run stores a
    Parquet copy (cache).
94      New/changed Excel files are detected
    automatically (-> cache is rebuilt).
95      """
96      cache = data_dir / "_energy_raw.parquet"
97      files = sorted(glob.glob(str(data_dir / "*
    report_by_device*.xlsx")))
98      if not files:
99          raise FileNotFoundError(
100             f"No '*report_by_device*.xlsx' files
    found in {data_dir}.\n"
101             f"-> Put the Excel files there or
    change DATA_DIR at the top of the script.")
102
103     # Use the cache only if it is newer than every
     Excel file
104     if cache.exists() and all(cache.stat().
    st_mtime > os.path.getmtime(f) for f in files):
105         print(f"[load] using cache: {cache.name}")
```

Saves PNG at 130dpi

fast binary form

all matching Excel files

n° files

Error Message

last modified time → changed Excel file: re-read automatically

```
106         return pd.read_parquet(cache)
107
108     frames = []
109     for f in files:
110         # Extract the zone (NE, NW, SE, SW) from
    the file name
111         zone = re.search(r"Storey1_([A-Z]{2})_",
    os.path.basename(f)).group(1)
112         try:
113             sheets = pd.read_excel(f, sheet_name=
    None, engine="calamine")  # fast engine
114         except ImportError:
115             print("  python-calamine not installed
     -> using openpyxl (much slower)")
116             sheets = pd.read_excel(f, sheet_name=
    None)                          # fallback
117         # sheet_name=None returns a dictionary {
    sheet name: DataFrame}
118         for sheet, df in sheets.items():
119             df = df.copy()
120             df["Zone"] = zone
121             df["Category"] = sheet.strip()
122             frames.append(df)
123             print(f"[load] {zone:2s} | {sheet:16s}
     | {len(df):>7,} rows")
124
125     raw = pd.concat(frames, ignore_index=True
    )       # stack everything vertically
126
127     # Enforce data types: invalid entries become
    NaT/NaN instead of crashing
128     raw["Timestamp"] = pd.to_datetime(raw["
    Timestamp"], errors="coerce")
129     for c in ENERGY_COLS + SENSOR_COLS + ["Number
    of Edevices"]:
130         raw[c] = pd.to_numeric(raw[c], errors="
    coerce")
131
132     # 'E Device type' is identical to the sheet
    name -> redundant
133     raw = raw.drop(columns=["E Device type"],
```

```
133 errors="ignore")
134     raw = raw.sort_values(["Zone", "Category", "
    Timestamp"]).reset_index(drop=True)
135     raw.to_parquet(cache)
136     return raw
137
138
139 # ================================================
    ============================
140 # 2) DATA QUALITY: overview per series (zone x
    category)
141 # ================================================
    ============================
142 def quality_overview(df: pd.DataFrame) -> pd.
    DataFrame:
143     """One row per series with time range, number
    of devices, missing values,
144     duplicates and the share of negative / zero
    power readings."""
145     g = df.groupby(["Zone", "Category"])
146     q = pd.DataFrame({
147         "rows": g.size(),
148         "start": g["Timestamp"].min(),
149         "end": g["Timestamp"].max(),
150         "n_devices": g["Number of Edevices"].first
    (),
151         "bad_timestamps": g["Timestamp"].apply(
    lambda s: s.isna().sum()),
152         "duplicate_timestamps": g["Timestamp"].
    apply(lambda s: s.duplicated().sum()),
153         "missing_values": g[ENERGY_COLS].apply(
    lambda x: x.isna().sum().sum()),
154         "W_negative_%": g["W"].apply(lambda s: (s
     < 0).mean() * 100).round(1),
155         "W_zero_%": g["W"].apply(lambda s: (s == 0
    ).mean() * 100).round(1),
156         # True if a series contains only zeros (
    meter dead / not connected)
157         "all_energy_zero": g[ENERGY_COLS].apply(
    lambda x: bool((x.fillna(0) == 0).all().all())),
158     })
```

Output

```
159     q.to_csv(OUT_DIR / "01_quality_overview.csv")
160     print("\n=== DATA QUALITY ===\n", q.to_string
    ())
161
162     # Turn the table into written findings
163     for (z, c), r in q.iterrows():
164         if r["all_energy_zero"]:
165             note(f"{z} | {c}: contains only zeros
     (inactive meter) -> removed from figures.")
166         elif r["W_negative_%"] > 90:
167             note(f"{z} | {c}: {r['W_negative_%']}
    % negative power -> probably inverted sign.")
168     last_end = q["end"].max()
169     for (z, c), r in q.iterrows():
170         if r["end"] < last_end - pd.Timedelta(days
    =30):
171             note(f"{z} | {c}: data already ends on
     {r['end']:%Y-%m-%d}.")
172     return q
173
174
175 def prepare(df: pd.DataFrame, q: pd.DataFrame) ->
    pd.DataFrame:
176     """IMPROVEMENT: prepare series for analysis (
    raw data stays unchanged).
177     - remove series without any readings (zeros
    only)
178     - series with almost only negative values ->
    flip the sign (meter wired the
179       wrong way round). This is an EDA assumption
    and must be agreed with Node A! ?
180     """
181     df = df.copy()
182     dead = q.index[q["all_energy_zero"]]
183     flipped = q.index[(q["W_negative_%"] > 90) & ~
    q["all_energy_zero"]]
184     key = list(zip(df["Zone"], df["Category"]))
185     df = df[~pd.Series(key, index=df.index).isin(
    dead)]
186     mask = pd.Series(list(zip(df["Zone"], df["
    Category"])), index=df.index).isin(flipped)
```

turns table into text

series that end >30days before the latest data = finding

cleaning (works on copy)

zeros only

mostly negative

AND NOT dead

builds (zone, category) tuples

finds them

→ dead series removed

rows belong to flip series

```
187     df.loc[mask, ["W", "Pa"]] = -df.loc[mask, ["W"
    , "Pa"]]    W & Pa *-1 = assumption
188     return df
189
190
191 # ================================================
    ============================
192 # 3) SAMPLING RATE & GAPS (consequence of the data
     retention policy)
193 # ================================================
    ============================
194 def sampling_analysis(df: pd.DataFrame):
195     """How often was data recorded and where is
    data missing?"""
196     df = df.copy()
197     # Time difference to the previous reading of
    the same series (minutes)
198     df["dt_min"] = df.groupby(["Zone", "Category"
    ])["Timestamp"].diff().dt.total_seconds() / 60
199     df["year"] = df["Timestamp"].dt.year
200
201     # Most frequent interval per year (= typical
    sampling rate)
202     freq = (df.groupby("year")["dt_min"]
203                 .agg(lambda s: s.mode().iloc[0] if s
    .notna().any() else np.nan)
204                 .rename("typical_interval_min"))
205     fy = pd.concat([df.groupby("year").size().
    rename("rows"), freq], axis=1)
206     fy.to_csv(OUT_DIR / "02_sampling_by_year.csv")
207     print("\n=== SAMPLING INTERVAL PER YEAR ===\n"
    , fy.to_string())
208     note("Sampling interval: " + ", ".join(f"{int(
    y)}: {v:.0f} min" for y, v in freq.items()))
209
210     # Gaps: intervals longer than
    GAP_THRESHOLD_MIN (180)
211     gaps = df[df["dt_min"] > GAP_THRESHOLD_MIN][["
    Zone", "Category", "Timestamp", "dt_min"]].copy()
212     gaps["gap_start"] = gaps["Timestamp"] - pd.
    to_timedelta(gaps["dt_min"], unit="min")
```

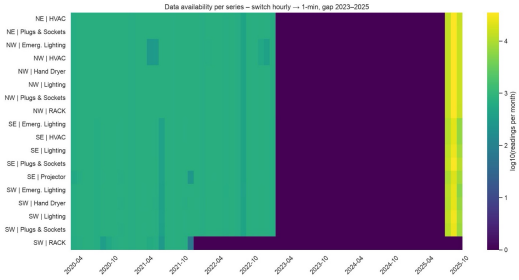
groupby, HVAC compared to lightning

most frequent interval per year

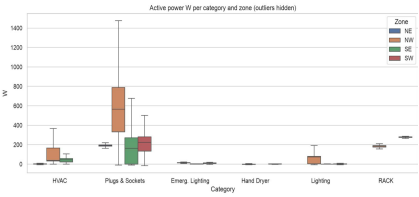
Output

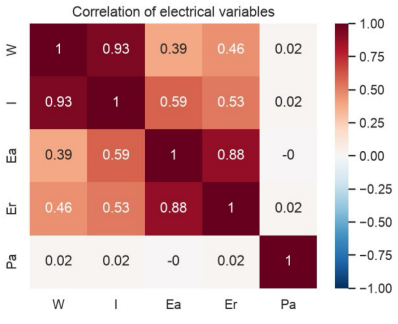
```
213     gaps["gap_days"] = (gaps["dt_min"] / 1440).
    round(2)
214     gaps = gaps.rename(columns={"Timestamp": "
    gap_end"})[["Zone", "Category", "gap_start", "
    gap_end", "gap_days"]]
215     gaps = gaps.sort_values("gap_days", ascending=
    False)
216     gaps.to_csv(OUT_DIR / "03_gaps.csv", index=
    False)
217     print(f"\n=== GAPS > {GAP_THRESHOLD_MIN} min:
    {len(gaps)} (10 largest) ===\n",
218           gaps.head(10).to_string(index=False))
219     if len(gaps):
220         g0 = gaps.iloc[0]
221         note(f"Largest gap: {g0.gap_start:%Y-%m-%d
    } to {g0.gap_end:%Y-%m-%d} ({g0.gap_days:.0f} days
    ).")
222
223     # IMPROVEMENT: time zone check. Is there a
    missing or duplicated hour on
224     # daylight-saving days? If not, the timestamps
     are most likely UTC.
225     hourly = df[df["dt_min"] == 60]
226     dst_days = [d for d in pd.to_datetime(["2021-
    03-28", "2022-03-27"])
227                 if d in set(hourly["Timestamp"].dt
    .normalize())]
228     if dst_days:
229         s = df[(df["Timestamp"].dt.normalize() ==
    dst_days[0])].groupby(["Zone", "Category"]).size()
230         note(f"DST change {dst_days[0]:%Y-%m-%d}:
    {int(s.median())} readings/day instead of 23 "
231              f"-> timestamps are probably UTC (
    important for Node B's weather merge).")
232
233     # Figure: data availability (readings per
    month, log scale)
234     avail = (df.assign(month=df["Timestamp"].dt.
    to_period("M").dt.to_timestamp(),
235                        series=df["Zone"] + " | "
     + df["Category"])
```

```
236                     .groupby(["series", "month"]).size
    ().unstack(fill_value=0))
237     # insert empty months so the big gap becomes
    visible
238     avail = avail.reindex(columns=pd.date_range(
    avail.columns.min(), avail.columns.max(), freq="MS
    "),
239                           fill_value=0)
240     fig, ax = plt.subplots(figsize=(14, 7))
241     sns.heatmap(np.log10(avail + 1), cmap="viridis
    ", ax=ax,
242                 cbar_kws={"label": "log10(readings
     per month)"})
243     ax.set_xticks(np.arange(0, avail.shape[1], 6
    ) + 0.5)
244     ax.set_xticklabels([c.strftime("%Y-%m") for c
    in avail.columns[::6]], rotation=45)
245     ax.set_title("Data availability per series -
    switch hourly → 1-min, gap 2023-2025")
246     ax.set_xlabel(""); ax.set_ylabel("")
247     finish(fig, "fig01_availability.png")
248
249
250 # ============================
    ============================
251 # 4) DESCRIPTIVE STATISTICS
252 # ============================
    ============================
253 def descriptive_stats(df: pd.DataFrame):
254     """Mean, spread, percentiles and distributions
     of the measured quantities."""
255     desc = df.groupby(["Zone", "Category"])[
    ENERGY_COLS].describe(percentiles=[.01, .5, .99]).
    round(2)
256     desc.to_csv(OUT_DIR / "04_descriptive_stats.
    csv")
257
258     # Boxplot of power per category and zone (
    outliers hidden for readability)
259     fig, ax = plt.subplots(figsize=(12, 5))
260     sns.boxplot(data=df[df["W"].notna()], x="
```



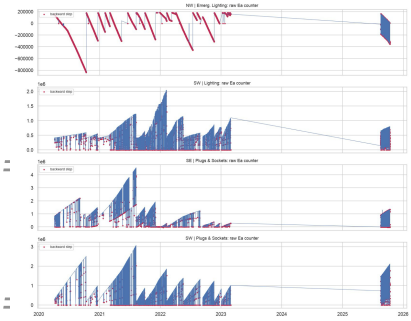
```
Category", y="W", hue="Zone", showfliers=False, ax=ax)
    ax.set_title("Active power W per category and zone (outliers hidden)")
    ax.set_ylabel("W")
    finish(fig, "fig02_power_boxplot.png")

    # Correlation between the electrical quantities
    corr = df[ENERGY_COLS].corr().round(2)
    corr.to_csv(OUT_DIR / "05_correlation.csv")
    fig, ax = plt.subplots(figsize=(5, 4))
    sns.heatmap(corr, annot=True, cmap="RdBu_r", vmin=-1, vmax=1, ax=ax)
    ax.set_title("Correlation of electrical variables")
    finish(fig, "fig03_correlation.png")


# ============================================================================
# 5) COUNTER RESETS & CONSISTENCY of Ea
# ============================================================================
def reset_analysis(df: pd.DataFrame):
    """Ea is a cumulative counter and should only increase. Every backward step
    is an anomaly (device reset, overflow or transmission error)."""
    df = df.copy()
    df["dEa"] = df.groupby(["Zone", "Category"])["Ea"].diff()   # change vs. previous reading
    df["ea_drop"] = df["dEa"] < 0                               # any backward step
    df["reset_to_zero"] = df["ea_drop"] & (df["Ea"] <= 1)       # drop to ~0 = real reset
    df["ea_negative"] = df["Ea"] < 0                            # physically impossible
    r = df.groupby(["Zone", "Category"]).agg(
        rows=("Ea", "size"),
```





```
287         ea_backward_steps=("ea_drop", "sum"),
288         resets_to_zero=("reset_to_zero", "sum"),
289         ea_negative_values=("ea_negative", "sum"),
290     )
291     r["backward_%"] = (r["ea_backward_steps"] / r[
    "rows"] * 100).round(1)
292     r.to_csv(OUT_DIR / "06_counter_resets.csv")
293     print("\n=== Ea COUNTER ANOMALIES ===\n", r.
    to_string())
294     note(f"Ea steps backwards {int(r['
    ea_backward_steps'].sum()):,} times in total "
295          f"({int(r['resets_to_zero'].sum()):,} of
    them to ~0) -> Ea is unreliable as a consumption
    source.")
296
297     # IMPROVEMENT: does Ea match W at all? For
    hourly data the following should hold:
298     # increase of Ea within one hour ≈ mean power
    W (in Wh or kWh).
299     df["dt_h"] = df.groupby(["Zone", "Category"])[
    "Timestamp"].diff().dt.total_seconds() / 3600
300     ok = df[(df["dt_h"] == 1) & (df["dEa"] > 0
    ) & (df["W"] > 0)]
301     cons = ok.groupby(["Zone", "Category"]).apply(
302         lambda s: pd.Series({"corr_dEa_W": s["dEa"
    ].corr(s["W"]),
303                              "
    median_ratio_dEa_per_W": (s["dEa"] / s["W"]).
    median()})).round(3)
304     cons.to_csv(OUT_DIR / "06b_ea_vs_w_consistency
    .csv")
305     print("\n=== CONSISTENCY hourly Ea increase vs
    . W ===\n", cons.to_string())
306     note(f"Correlation of hourly Ea increase vs. W
    : median {cons['corr_dEa_W'].median():.2f} "
307          f"-> Ea and W do not match; units of Ea
    must be clarified. Consumption is therefore
    derived from W.")
308
309     # Figure: raw Ea for the 4 most affected
    series, backward steps in red
```

```
310     worst = r["ea_backward_steps"].sort_values(
    ascending=False).head(4).index
311     fig, axes = plt.subplots(len(worst), 1,
    figsize=(13, 2.6 * len(worst)), sharex=True)
312     for ax, (z, c) in zip(np.atleast_1d(axes),
    worst):
313         s = df[(df.Zone == z) & (df.Category == c
    )]
314         ax.plot(s["Timestamp"], s["Ea"], lw=0.6)
315         drops = s[s["ea_drop"]]
316         ax.scatter(drops["Timestamp"], drops["Ea"
    ], s=4, color="crimson", label="backward step")
317         ax.set_title(f"{z} | {c}: raw Ea counter"
    , fontsize=10)
318         ax.legend(loc="upper left", fontsize=8)
319     finish(fig, "fig04_ea_resets.png")
320
321
322 # ========================================
    ============================
323 # 6) HOURLY TIME SERIES & TIME PATTERNS
324 # ========================================
    ============================
325 def to_hourly(df: pd.DataFrame) -> pd.DataFrame:
326     """Bring hourly (2020-23) and 1-minute (2025)
    data onto a common basis:
327     the mean per hour. Mean power W over 1 h =
    energy in Wh for that hour.
328     'n_obs' counts how many raw readings went into
     each hour."""
329     h = (df.set_index("Timestamp")
330             .groupby(["Zone", "Category"])
331             .resample("1h")
332             .agg(W=("W", "mean"), I=("I", "mean"),
    Pa=("Pa", "mean"), n_obs=("W", "count"))
333             .reset_index())
334     h = h[h["n_obs"] > 0]                  # drop
    hours without any reading (e.g. gap 2023-25)
335     h["W_pos"] = h["W"].clip(lower=0)      # small
    negative values = measurement noise -> 0
336     h.to_parquet(OUT_DIR / "hourly_energy.parquet"
```



most backwards step

same as loop also work without subplot

output

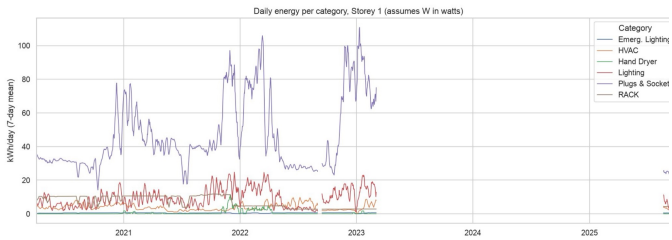
1 hour blocks & take the mean

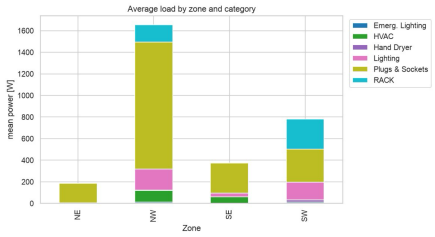
drops empty hours

negative values set to 0

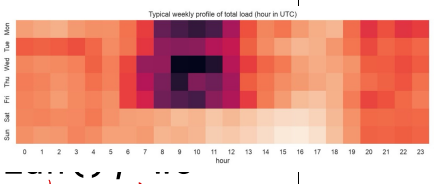
Wh in that hour

```
336 )
337     return h
338
339
340 def time_patterns(h: pd.DataFrame):
341     """Typical patterns: daily energy, shares,
    weekly/daily profile, seasonality, distribution
    ."""
342     h = h.copy()
343
344     # 6a) Daily energy per category (sum over all
    zones).
345     # IMPROVEMENT: only days with enough hours,
    otherwise incomplete days look like drops.
346     h["day"] = h["Timestamp"].dt.floor("D")
347     hours_per_day = h.groupby(["Zone", "Category"
    , "day"])["Timestamp"].transform("count")
348     daily = (h[hours_per_day >= MIN_HOURS_PER_DAY]
349     .groupby(["Category", "day"])["
    W_pos"].sum().div(1000).rename("kWh").reset_index
    ())
350     piv = daily.pivot(index="day", columns="
    Category", values="kWh")
351     piv = piv.reindex(pd.date_range(piv.index.min
    (), piv.index.max(), freq="D"))  # keep gaps
    visible
352     fig, ax = plt.subplots(figsize=(14, 5))
353     piv.rolling(7, min_periods=3).mean().plot(ax=
    ax, lw=1)
354     ax.set_ylabel("kWh/day (7-day mean)")
355     ax.set_title("Daily energy per category,
    Storey 1 (assumes W in watts)")
356     finish(fig, "fig05_daily_energy.png")
357
358     # 6b) Average load p
    category
359     share = h.groupby(["
    "].mean().unstack().fill
360     fig, ax = plt.subplo
361     share.plot(kind="bar", stacked=True, ax=ax,
    colormap="tab10")
```






```
362     ax.set_ylabel("mean power [W]"); a
    "Average load by zone and category")
363     ax.legend(bbox_to_anchor=(1.01, 1)
     left")
364     finish(fig, "fig06_load_share.png")
365     cat_mean = h.groupby("Category")["W_pos"].mean
    ()
366     tot_share = cat_mean / cat_mean.sum() * 100
367     note("Load shares: " + ", ".join(f"{c} {v:.0f}
    %" for c, v in tot_share.sort_values(ascending=
    False).items()))
368
369     # 6c) Weekly profile (hour x weekday) of the
    total load
370     tot = h.groupby("Timestamp")["W_pos"].sum().
    to_frame("W")
371     tot["hour"] = tot.index.hour
372     tot["weekday"] = tot.index.dayofweek
    # 0 = Monday ... 6 = Sunday
373     prof = tot.pivot_table(index="weekday",
    columns="hour", values="W", aggfunc="median")
374     prof.index = ["Mon", "Tue", "Wed", "Thu", "Fri
    ", "Sat", "Sun"][:len(prof)]
375     fig, ax = plt.subplots(figsize=(13, 4))
376     sns.heatmap(prof, cmap="rocket_r", ax=ax,
    cbar_kws={"label": "median W"})
377     ax.set_title("Typical weekly profi
    load (hour in UTC)")
378     finish(fig, "fig07_weekly_profile.
379     wk = tot[tot.weekday < 5]["W"].med
    tot[tot.weekday >= 5]["W"].median()
380     note(f"Median total load weekday {wk:.0f} W vs
    . weekend {we:.0f} W (+{(wk / we - 1) * 100:.0f}
    %).")
381
382     # 6d) Daily profile per category: weekday vs.
    weekend
383     h["daytype"] = np.where(h["Timestamp"].dt.
    dayofweek < 5, "weekday", "weekend")
384     h["hour"] = h["Timestamp"].dt.hour
385     cats = sorted(h["Category"].unique())
```



```
386     ncol = int(np.ceil(len(cats) / 2))
387     fig, axes = plt.subplots(2, ncol, figsize=(4
     * ncol, 6), sharex=True)
388     for ax, c in zip(axes.flat, cats):
389         p = h[h.Category == c].groupby(["daytype"
    , "hour"])["W_pos"].median().unstack(0)
390         p.plot(ax=ax, lw=1.5); ax.set_title(c,
    fontsize=10); ax.set_xlabel("hour (UTC)")
391     for ax in list(axes.flat)[len(cats):]:
392         ax.axis("off")
    # hide empty panels
393     fig.suptitle("Median daily load prof
    weekday vs. weekend")
394     finish(fig, "fig08_daily_profiles.pn
395
396     # 6e) Seasonality: mean load per calendar
    month
397     mon = h.groupby([h["Timestamp"].dt.month.
    rename("month"), "Category"])["W_pos"].mean().
    unstack()
398     fig, ax = plt.subplots(figsize=(11, 4))
399     mon.plot(ax=ax, marker="o"); ax.set_ylabel("
    mean W"); ax.set_title("Seasonality: mean load per
     month")
400     ax.set_xticks(range(1, 13))
401     ax.legend(bbox_to_anchor=(1.01, 1), loc="upper
     left")
402     finish(fig, "fig09_monthly.png"
403
404     # 6f) NEW: distribution of hour
    category (log scale due to wide range)
405     fig, ax = plt.subplots(figsize=(11, 4))
406     for c in cats:
407         v = h.loc[(h.Category == c) & (h.W_pos > 0
    ), "W_pos"]
408         if len(v):
409             ax.hist(v, bins=np.logspace(-1, 4, 60
    ), histtype="step", lw=1.5, label=c)
410     ax.set_xscale("log"); ax.set_xlabel("hourly
    power W (log)"); ax.set_ylabel("number of hours")
411     ax.set_title("Distribution of hourly power per
```

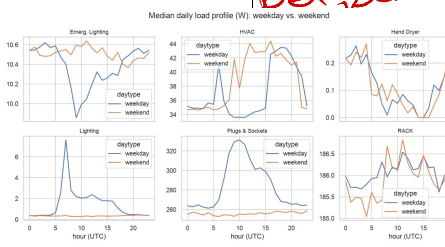
grid with 2 rows

flat list

median per h, split into weekday weekend

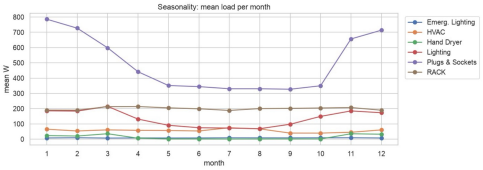
hide unused panel → 2 lines per panel

output



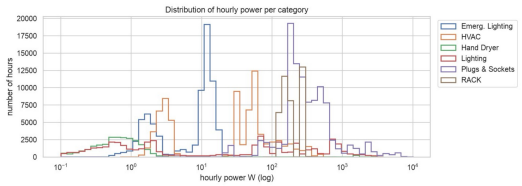
(1-24) all years, HVAC peaks expected winter/summer

output



Histogram with log-spaced bins 0.1 W to 10,000 W

outline only



output

```
411     category")
412         ax.legend(bbox_to_anchor=(1.01, 1), loc="upper
        left")
413         finish(fig, "fig10_distribution.png")
414
415         # 6g) NEW: autocorrelation of total load -
    reveals daily (24 h) and weekly (168 h) rhythm.
416         # Relevant for Sprint 2: which lags are good
    features for forecasting models?
417         s = tot["W"].asfreq("1h")  # regular hourly grid (gaps = NaN)
418         lags = np.arange(1, 24 * 14 + 1)
419         acf = [s.autocorr(lag=int(l)) for l in lags]
420         fig, ax = plt.subplots(figsize=(12, 3.5))
421         ax.plot(lags, acf, lw=1.2)
422         for x in (24, 168, 336):
423             ax.axvline(x, color="grey", ls="--", lw=0.
    8)
424         ax.set_xlabel("lag [hours]"); ax.set_ylabel("
    autocorrelation")
425         ax.set_title("Autocorrelation of total load (
    dashed: 1 day, 1 week, 2 weeks)")
426         finish(fig, "fig11_autocorrelation.png")
427         note(f"Autocorrelation of total load: lag 24 h
     = {acf[23]:.2f}, lag 168 h = {acf[167]:.2f} "
428              f"-> strong daily/weekly patterns, good
    lag features for Sprint 2.")
429
430
431 # ====================
    ========================
432 # 7) OUTLIERS (robust z-score on hourly power)
433 # ============================================
    ============================
434 def outliers(h: pd.DataFrame):
435     """Robust z-score = (value - median) / (1.4826
     * MAD).
436     Unlike mean/std, median and MAD are barely
    distorted by the outliers themselves."""
437     def robust_z(s):
438         # Use operating hours only (W > 0) as
```

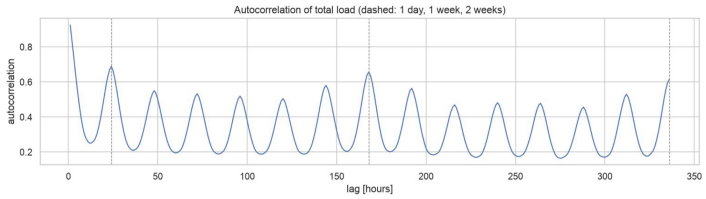
how similar the series is to itself shifted by k hours

regular hourly grid

lags 1 to 336h (2 weeks)

dashed lines day, week, 336

Output



```
438 reference: for on/off loads such as
439         # lighting the median would otherwise be 0
     and EVERY 'on' hour an outlier.
440         on = s[s > 0]
441         if len(on) < 10:
442             return s * 0
443         med = on.median()
444         mad = (on - med).abs().median()
    # median absolute deviation
445         z = (s - med) / (1.4826 * mad) if mad > 0
    else s * 0
446         return z.where(s > 0, 0)
    # 'off' hours are not outliers
447     h = h.copy()
448     h["rz"] = h.groupby(["Zone", "Category"])["W"
    ].transform(robust_z)
449     o = (h.assign(outlier=h["rz"].abs() >
    OUTLIER_Z)
450             .groupby(["Zone", "Category"])["outlier
    "].agg(["sum", "mean"]))
451     o.columns = ["outlier_hours", "outlier_share_
    %"]
452     o["outlier_share_%"] = (o["outlier_share_%"
    ] * 100).round(2)
453     o.to_csv(OUT_DIR / "07_outliers_hourly.csv")
454     print(f"\n=== OUTLIER HOURS (|robust z| > {
    OUTLIER_Z}) ===\n", o.to_string())
455     top = o["outlier_share_%"].idxmax()
456     note(f"Most outliers: {top[0]} | {top[1]} ({o.
    loc[top, 'outlier_share_%']}% of hours) "
457          f"- mostly real winter peaks, do not
    delete blindly.")
458
459
460 # ================================================
    ============================
461 # MAIN PROGRAM
462 # ================================================
    ============================
463 def main():
464     OUT_DIR.mkdir(exist_ok=True)
```

```
465     print(f"Reading Excel files from: {DATA_DIR}
      (first run ~30 s, then cached)")
466
467     df = load_all(DATA_DIR)
468     print(f"\n{len(df):,} rows | {df.Zone.nunique
    ()} zones | {df.Category.nunique()} categories")
469     print(f"Time range: {df.Timestamp.min()} → {df
    .Timestamp.max()}")
470     note("Note: data is aggregated per zone and
    device category (column 'Number of Edevices'), "
471          "not per individual meter.")
472
473     q = quality_overview(df)          # 2) quality
    of the raw data
474     sampling_analysis(df)             # 3) sampling
    rate, gaps, time zone
475     df_clean = prepare(df, q)         #    drop dead
     series, fix sign
476     descriptive_stats(df_clean)       # 4)
    statistics
477     reset_analysis(df)                # 5) counter
    resets (on raw data!)
478     h = to_hourly(df_clean)           # 6) common
   hourly basis
479     time_patterns(h)                  #    time
    patterns
480     outliers(h)                       # 7) outliers
481
482     # Summary as text file - good starting point
    for the sprint report
483     (OUT_DIR / "00_key_findings.txt").write_text(
484         "KEY FINDINGS - Energy EDA (generated
    automatically)\n\n" +
485         "\n".join(f"- {f}" for f in FINDINGS),
    encoding="utf-8")
486     print(f"\nDone. Figures and tables in {OUT_DIR
    }/")
487
488     if SHOW_PLOTS:
489         # Show all finished figures in one go (
    only the final versions, no intermediate steps)
```

```
490         print(f"Showing {len(plt.get_fignums())}
    figures ...")
491         plt.show()
492
493
494 if __name__ == "__main__":
495     main()
496
```

W = watts
I = intensity
Pa = Power apparent
Ea = Energy active
Er = Energy reactive

Summary:

W, I, Pa = how fast energy being used right now

Ea, Er = meter reading ⇒ keeps adding up over time (difference between readings for consumption)

1. Load the data (everything big table)
2. Check quality: missing values, duplicates, only zeros, negative values)
3. Check when data recorded: every how, minute, gaps no data, UTC or local time
4. Clean data: remove dead meters, flip sign on reversed meters
5. Basis statistics: average, spread, relations
6. Check energy counter (Ea): should only go up
   > code finds where it jumps backwards tests whether it matches power (W) > it doesn't so consumption calculated from W instead
7. Make everything hourly for comparism
8. Find patterns: daily energy use, which zones, categories use most, weekdays vs weekend, time of day, seasons
9. Find outliers: Hours with unusually high/low usage
10. Write summary & plots & tables

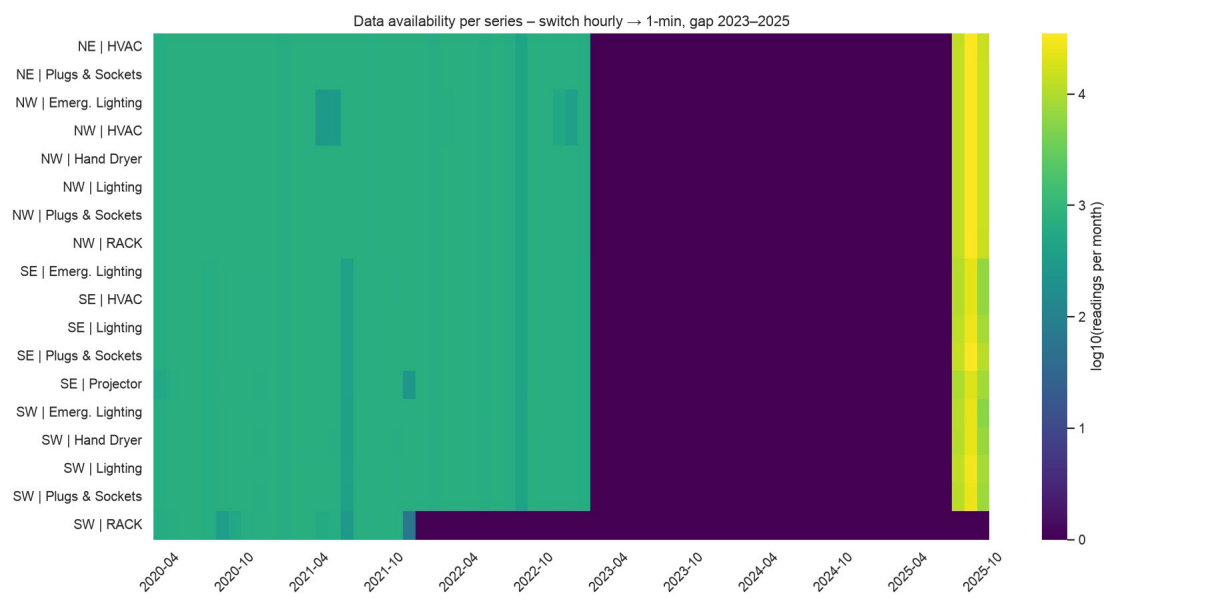
KEY FINDINGS - Energy EDA (generated automatically)

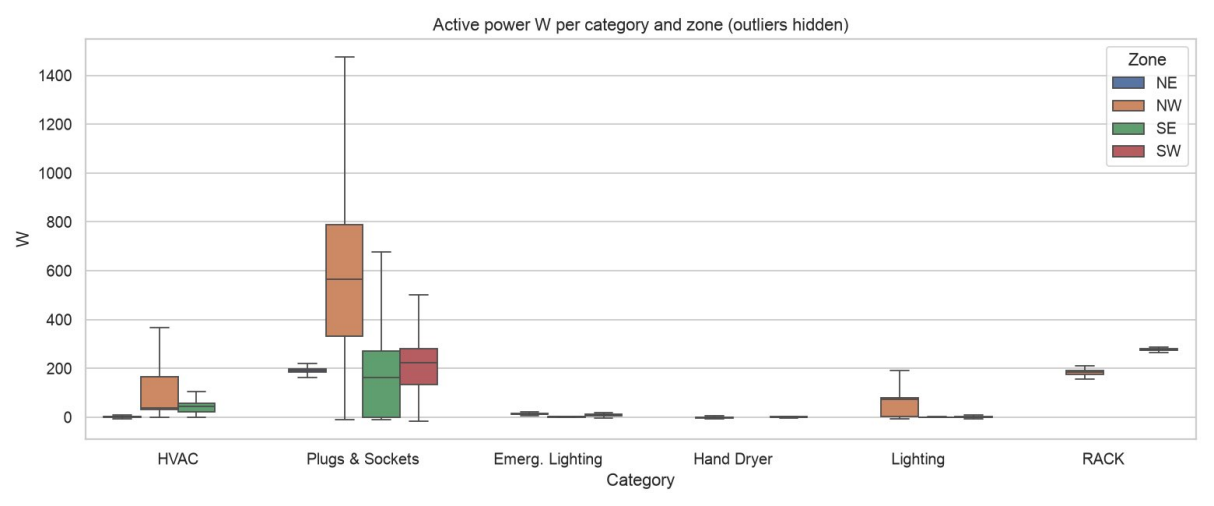
- Note: data is aggregated per zone and device category (column 'Number of Edevices'), not per individual meter.
- NW | Emerg. Lighting: 99.3% negative power -> probably inverted sign.
- SE | Projector: contains only zeros (inactive meter) -> removed from figures.
- SW | RACK: data already ends on 2021-12-03.
- Sampling interval: 2020: 60 min, 2021: 60 min, 2022: 60 min, 2023: 60 min, 2025: 1 min
- Largest gap: 2023-02-28 to 2025-08-18 (902 days).
- DST change 2021-03-28: 24 readings/day instead of 23 -> timestamps are probably UTC (important for Node B's weather merge).
- Ea steps backwards 256,774 times in total (87,210 of them to ~0) -> Ea is unreliable as a consumption source.
- Correlation of hourly Ea increase vs. W: median 0.03 -> Ea and W do not match; units of Ea must be clarified. Consumption is therefore derived from W.
- Load shares: Plugs & Sockets 54%, RACK 22%, Lighting 15%, HVAC 6%, Hand Dryer 1%, Emerg. Lighting 1%
- Median total load weekday 2366 W vs. weekend 2157 W (+10%).
- Autocorrelation of total load: lag 24 h = 0.69, lag 168 h = 0.66 -> strong daily/weekly patterns, good lag features for Sprint 2.
- Most outliers: SW | Lighting (20.26% of hours) - mostly real winter peaks, do not delete blindly.

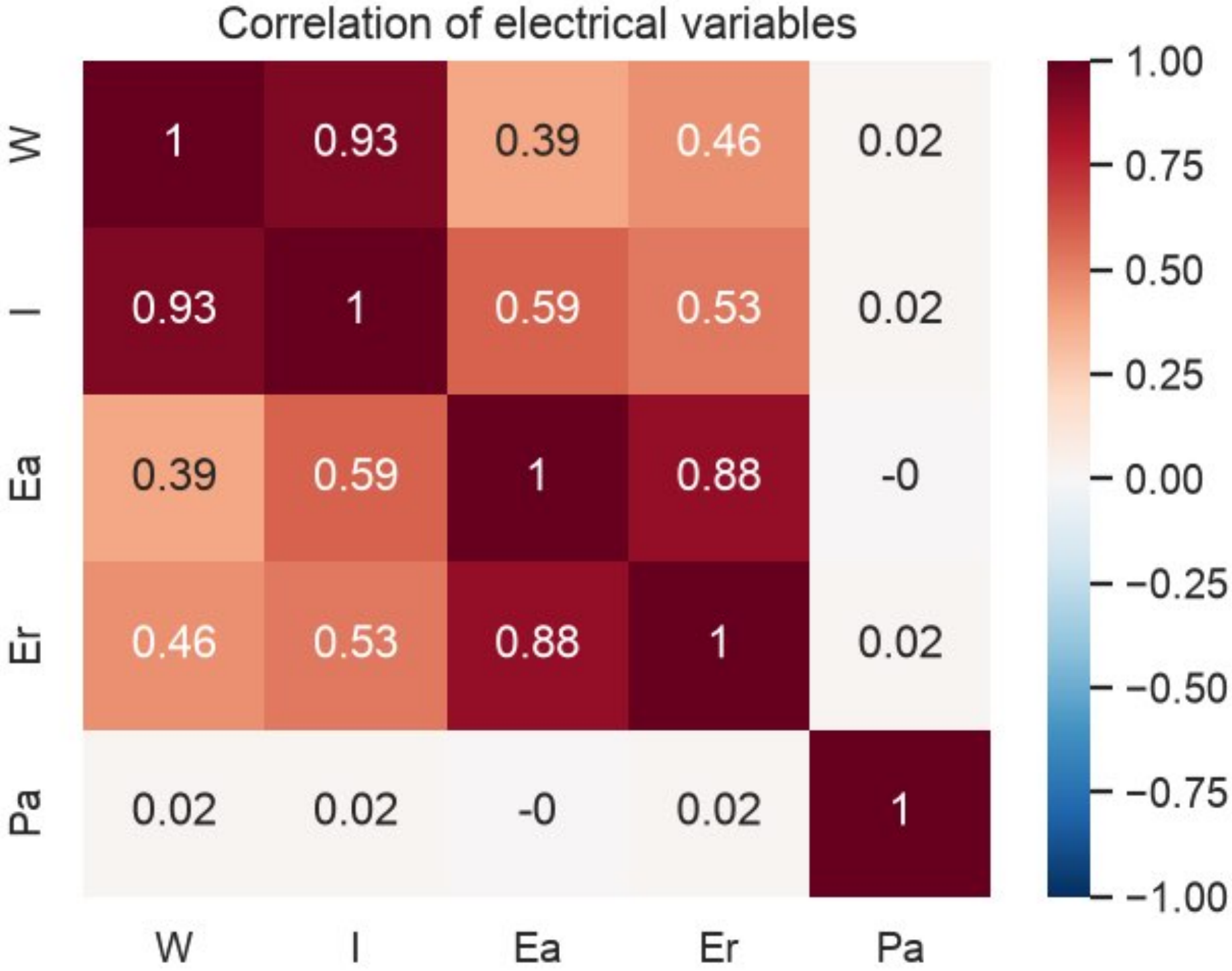
EDA Figures Overview

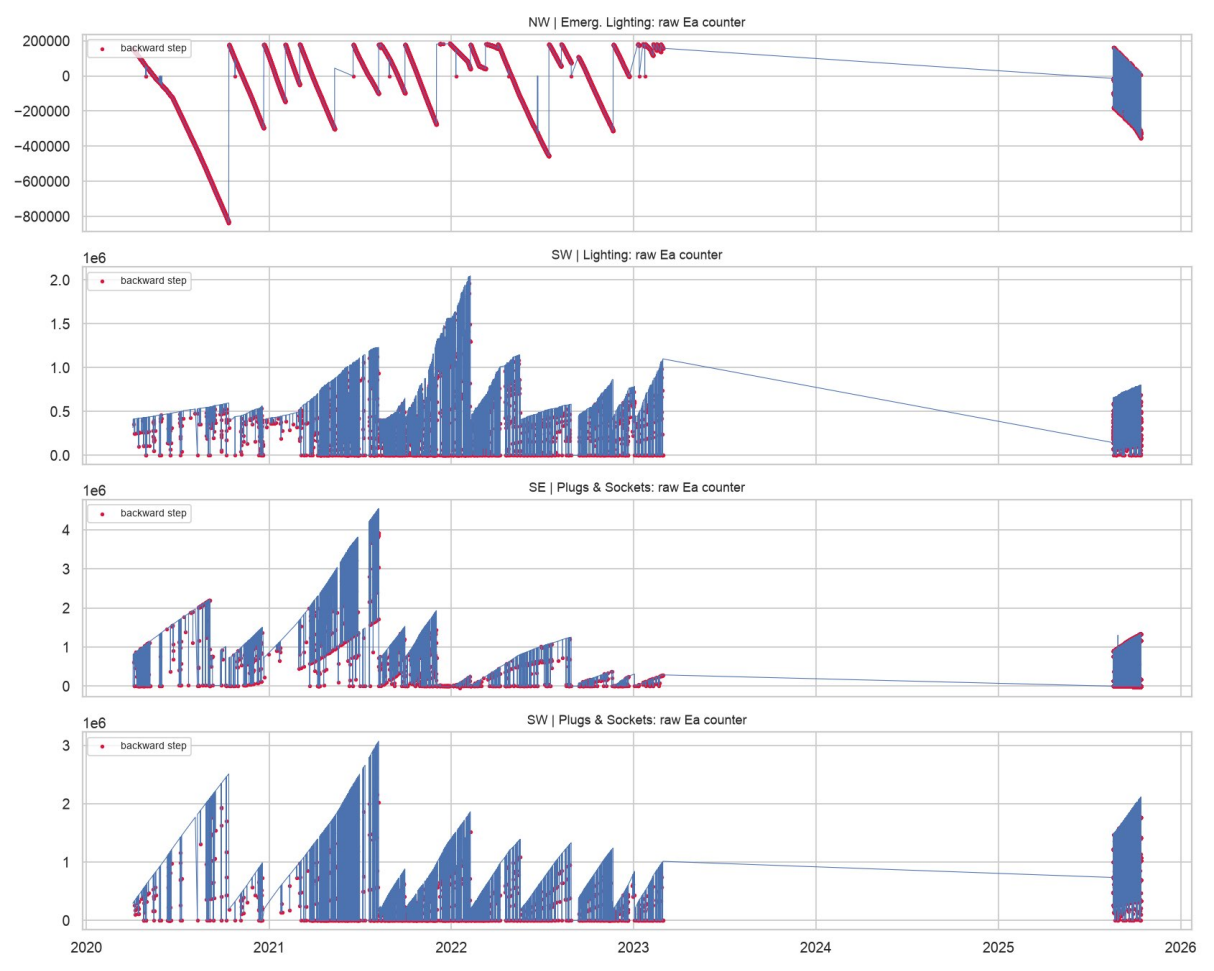
Sprint 1 – Task 5: Energy data, CeDInt building, Storey 1. All figures are saved in the folder *eda_output*.

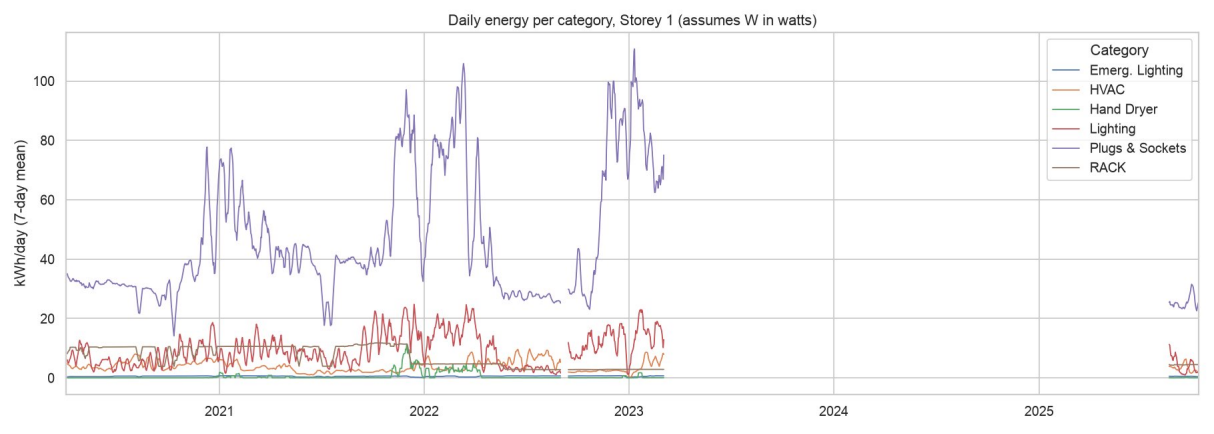
1	**fig01_availability.png**	A heatmap showing how many readings each series (zone × category) has per month, which makes the switch from hourly to 1-minute data and the 2023–2025 gap visible.
2	**fig02_power_boxplot.png**	A boxplot comparing the typical active power (W) of each device category, split by zone, with extreme outliers hidden.
3	**fig03_correlation.png**	A heatmap showing how strongly the five electrical variables (W, I, Ea, Er, Pa) are correlated with each other.
4	**fig04_ea_resets.png**	The raw Ea energy counter over time for the four most affected series, with red dots marking every point where the counter jumps backwards.
5	**fig05_daily_energy.png**	The daily energy consumption in kWh per device category over time, smoothed with a 7-day rolling mean.
6	**fig06_load_share.png**	A stacked bar chart showing the average power per zone, split into the share of each device category.
7	**fig07_weekly_profile.png**	A heatmap of the typical (median) total load for each weekday and hour of the day, showing when the building uses the most electricity.
8	**fig08_daily_profiles.png**	One small chart per device category comparing the typical daily load curve on weekdays and weekends.
9	**fig09_monthly.png**	The average power per calendar month for each device category, showing seasonal patterns such as higher heating or cooling demand.
10	**fig10_distribution.png**	Histograms on a log scale showing how often different hourly power levels occur in each device category.
11	**fig11_autocorrelation.png**	The autocorrelation of the total load for time shifts of up to two weeks, with peaks at 24 h and 168 h revealing daily and weekly rhythms.

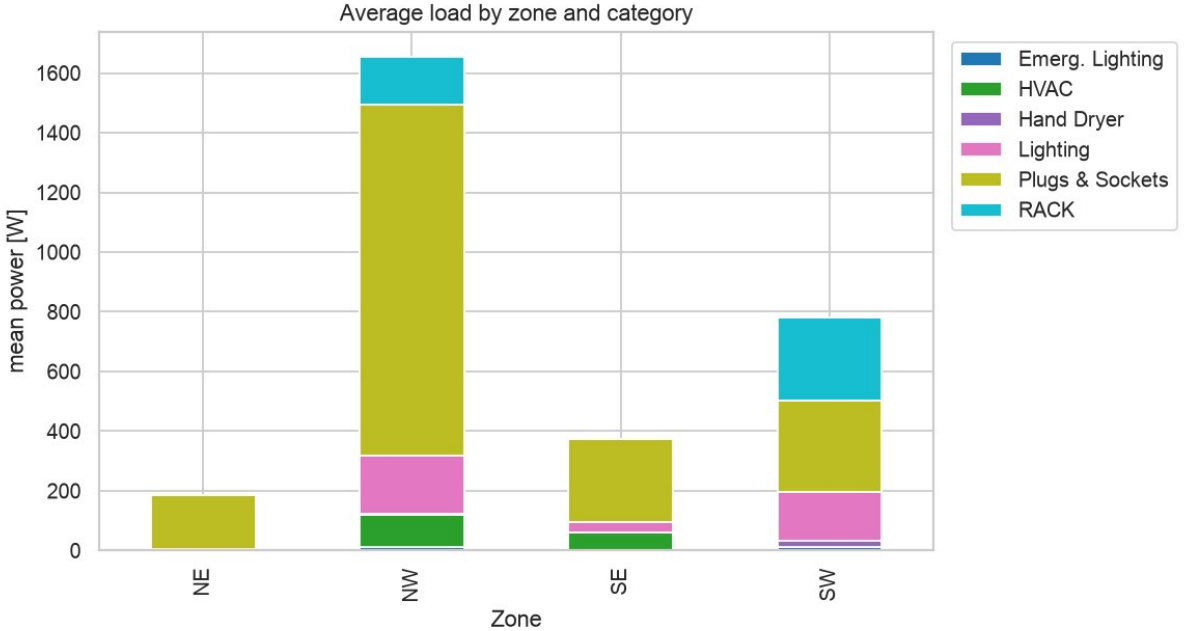

Data availability per series – switch hourly → 1-min, gap 2023–2025
NE | HVAC
NE | Plugs & Sockets
NW | Emerg. Lighting
NW | HVAC
NW | Hand Dryer
NW | Lighting
NW | Plugs & Sockets
NW | RACK
SE | Emerg. Lighting
SE | HVAC
SE | Lighting
SE | Plugs & Sockets
SE | Projector
SW | Emerg. Lighting
SW | Hand Dryer
SW | Lighting
SW | Plugs & Sockets
SW | RACK
2020-04
2020-10
2021-04
2021-10
2022-04
2022-10
2023-04
2023-10
2024-04
2024-10
2025-04
2025-10
log10(readings per month)
0
1
2
3
4

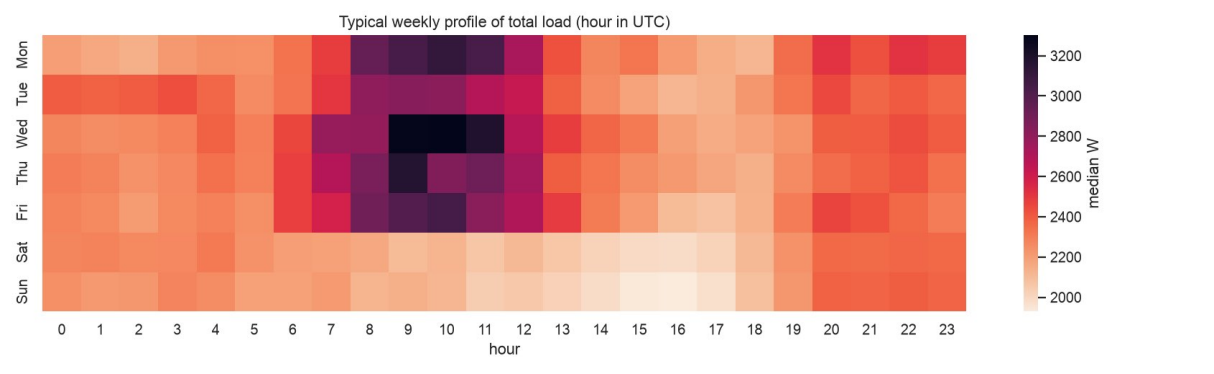

Active power W per category and zone (outliers hidden)
Zone
NE
NW
SE
SW
W
1400
1200
1000
800
600
400
200
0
HVAC
Plugs & Sockets
Emerg. Lighting
Hand Dryer
Lighting
RACK
Category

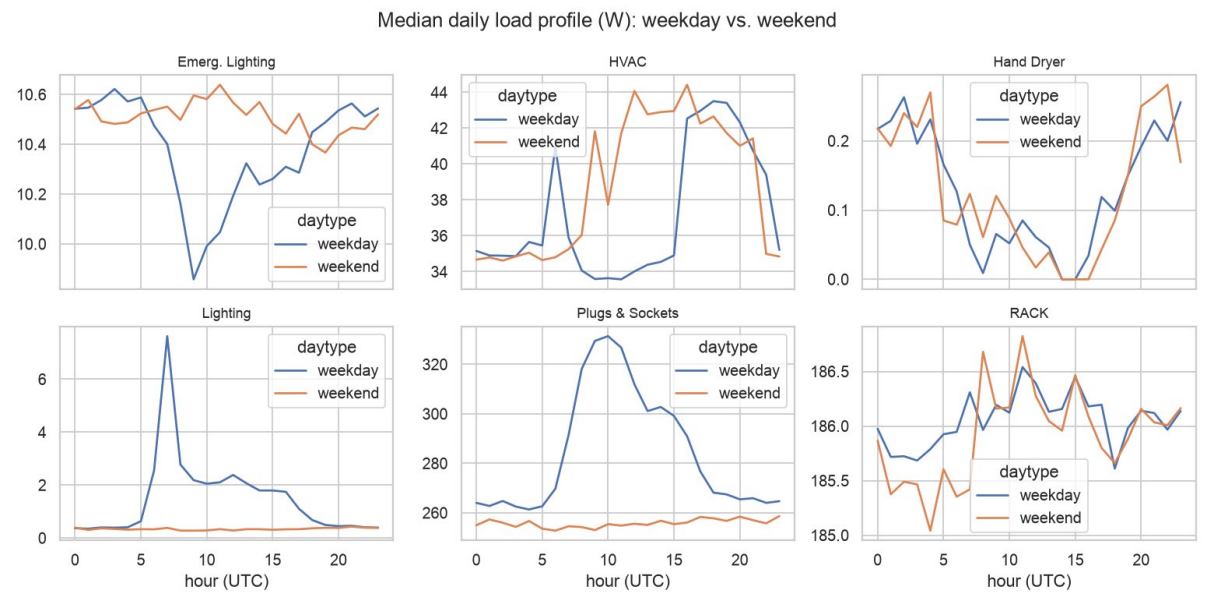

Correlation of electrical variables
W
I
Ea
Er
Pa
1
0.93
0.39
0.46
0.02
0.93
1
0.59
0.53
0.02
0.39
0.59
1
0.88
-0
0.46
0.53
0.88
1
0.02
0.02
0.02
-0
0.02
1
W
I
Ea
Er
Pa
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

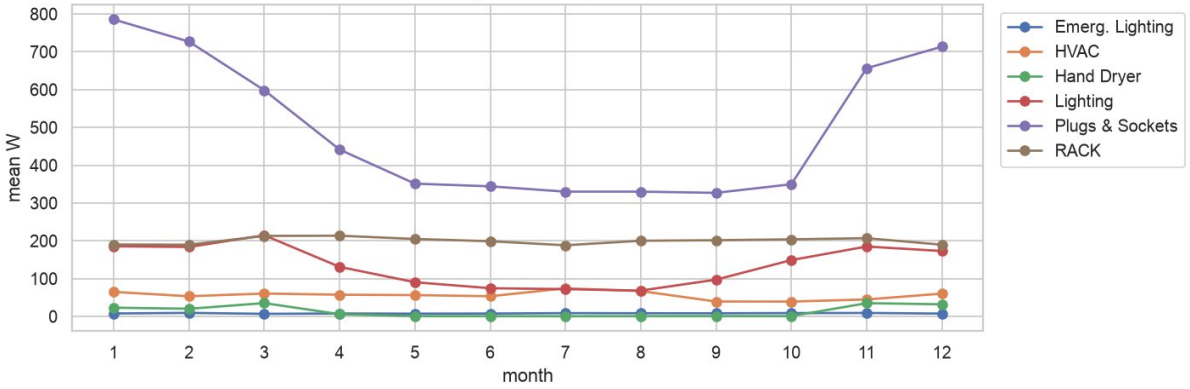

NW | Emerg. Lighting: raw Ea counter
backward step
200000
0
−200000
−400000
−600000
−800000
SW | Lighting: raw Ea counter
1e6
backward step
2.0
1.5
1.0
0.5
0.0
SE | Plugs & Sockets: raw Ea counter
1e6
backward step
4
3
2
1
0
SW | Plugs & Sockets: raw Ea counter
1e6
backward step
3
2
1
0
2020
2021
2022
2023
2024
2025
2026

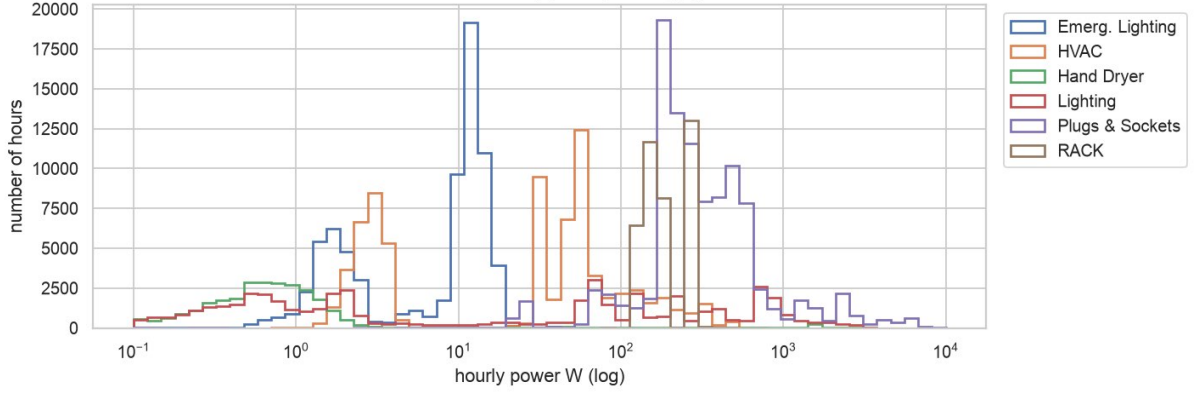

Daily energy per category, Storey 1 (assumes W in watts)
kWh/day (7-day mean)
0
20
40
60
80
100
2021
2022
2023
2024
2025
Category
Emerg. Lighting
HVAC
Hand Dryer
Lighting
Plugs & Sockets
RACK

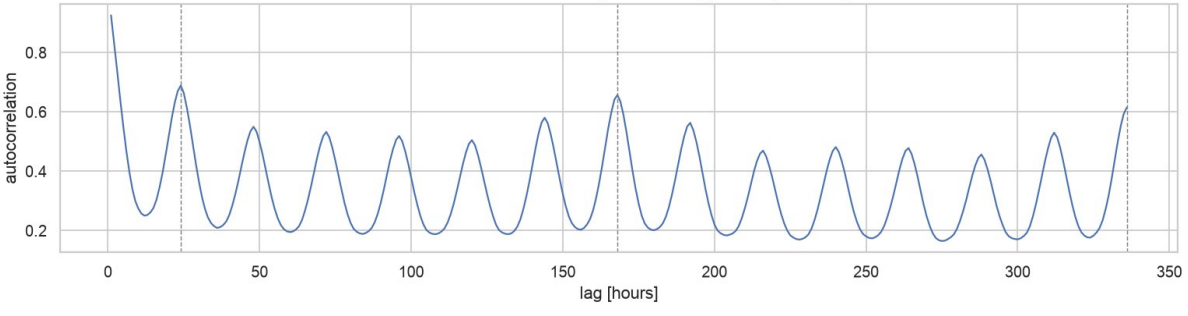

Average load by zone and category
mean power [W]
0
200
400
600
800
1000
1200
1400
1600
NE
NW
SE
SW
Zone
Emerg. Lighting
HVAC
Hand Dryer
Lighting
Plugs & Sockets
RACK


Typical weekly profile of total load (hour in UTC)
Mon
Tue
Wed
Thu
Fri
Sat
Sun
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
hour
3200
3000
2800
2600
2400
2200
2000
median W


Median daily load profile (W): weekday vs. weekend
Emerg. Lighting
HVAC
Hand Dryer
Lighting
Plugs & Sockets
RACK
daytype
weekday
weekend
hour (UTC)


Seasonality: mean load per month
mean W
month
Emerg. Lighting
HVAC
Hand Dryer
Lighting
Plugs & Sockets
RACK


Distribution of hourly power per category
number of hours
20000
17500
15000
12500
10000
7500
5000
2500
0
10^{-1}
10^{0}
10^{1}
10^{2}
10^{3}
10^{4}
hourly power W (log)
Emerg. Lighting
HVAC
Hand Dryer
Lighting
Plugs & Sockets
RACK


Autocorrelation of total load (dashed: 1 day, 1 week, 2 weeks)
autocorrelation
0.8
0.6
0.4
0.2
0
50
100
150
200
250
300
350
lag [hours]